



PDF Download
3428225.pdf
02 April 2026
Total Citations: 17
Total Downloads: 722

 Latest updates: <https://dl.acm.org/doi/10.1145/3428225>

RESEARCH-ARTICLE

Automatic and efficient variability-aware lifting of functional programs

RAMY SHAHIN, University of Toronto, Toronto, ON, Canada

MARSHA CHECHIK, University of Toronto, Toronto, ON, Canada

Open Access Support provided by:

University of Toronto

Published: 13 November 2020

[Citation in BibTeX format](#)

Automatic and Efficient Variability-Aware Lifting of Functional Programs

RAMY SHAHIN, University of Toronto, Canada

MARSHA CHECHIK, University of Toronto, Canada

A software analysis is a computer program that takes some representation of a software product as input and produces some useful information about that product as output. A software product line encompasses *many* software product variants, and thus existing analyses can be applied to each of the product variations individually, but not to the entire product line as a whole. Enumerating all product variants and analyzing them one by one is usually intractable due to the combinatorial explosion of the number of product variants with respect to product line features. Several software analyses (e.g., type checkers, model checkers, data flow analyses) have been redesigned/re-implemented to support variability. This usually requires a lot of time and effort, and the variability-aware version of the analysis might have new errors/bugs that do not exist in the original one.

Given an analysis program written in a functional language based on PCF, in this paper we present two approaches to transforming (lifting) it into a semantically equivalent variability-aware analysis. A light-weight approach (referred to as *shallow lifting*) wraps the analysis program into a variability-aware version, exploring all combinations of its input arguments. Deep lifting, on the other hand, is a program rewriting mechanism where the syntactic constructs of the input program are rewritten into their variability-aware counterparts. Compositionally this results in an efficient program semantically equivalent to the input program, modulo variability. We present the correctness criteria for functional program lifting, together with correctness proof sketches of shallow lifting. We evaluate our approach on a set of program analyses applied to the BusyBox C-language product line.

CCS Concepts: • **Software and its engineering** → *Automated static analysis; Software design techniques.*

Additional Key Words and Phrases: Software Product Lines, PCF, Program Rewriting, Lifting, Variability-aware Programming

ACM Reference Format:

Ramy Shahin and Marsha Chechik. 2020. Automatic and Efficient Variability-Aware Lifting of Functional Programs. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 157 (November 2020), 27 pages. <https://doi.org/10.1145/3428225>

1 INTRODUCTION

A *Software Product Line (SPL)* is a family of software products, developed together from a common set of artifacts. The unit of variability in an SPL is a *feature*, which can be either present or absent in a given product. A *product* is thus considered a variant, which can be generated from the SPL given a *feature configuration*. In *annotative* SPLs, feature-specific artifacts are annotated using feature expressions. For example, a commonly used approach for annotating the source code of an

Authors' addresses: Ramy Shahin, Computer Science Department, University of Toronto, Toronto, Canada, rshahin@cs.toronto.edu; Marsha Chechik, Computer Science Department, University of Toronto, Toronto, Canada, chechik@cs.toronto.edu.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

© 2020 Copyright held by the owner/author(s).

2475-1421/2020/11-ART157

<https://doi.org/10.1145/3428225>

SPL is C Pre-Processor (CPP) macro definitions and conditional compilation. Features are denoted by pre-processor macro definitions, and code segments are annotated by `#ifdef` blocks. Feature configurations are defined at build-time by passing macro definitions corresponding to the selected features to the CPP, which excludes the code of unselected features from being built.

Product Line Engineering (PLE) is considered much more economical in terms of time and effort compared to maintaining each of the different product variants separately [Clements and Northrop 2001]. For this reason, SPLs are used in many software domains that require a high degree of variability and configurability, including operating systems, embedded systems and compilers [Liebig et al. 2010; Pohl et al. 2005]. For example, the Linux operating system kernel is an SPL with more than 10,000 features [Nadi and Holt 2014].

A *Software Analysis Tool* is a program that takes software artifacts (e.g., source code) as input, and computes some useful information about it. Examples include syntax checkers (parsers), type checkers, software model checkers and other bug finding tools. While such tools and PLE are very widely used individually, it is challenging to use them together. In particular, given a software analysis tool G and an SPL $\mathcal{L}$, we would like to be able to apply G to all the product variants of $\mathcal{L}$ *simultaneously*, without having to enumerate the potentially exponential number of product variants and applying G to each. However, since G is designed to take the artifacts of a single product as input, this is not generally possible.

To get around the intractability of analyzing each possible product, some product sampling techniques have been suggested [Apel et al. 2013; Liebig et al. 2013]. Sampling tackles the combinatorial complexity of the problem by giving up completeness. The selected samples typically cover only a subset of possible feature interactions, which might miss many significant behaviors involving several interacting features.

Alternatively, some analyses have been rewritten to support full SPL analysis. These include syntax checkers (parsers) [Gazzillo and Grimm 2012; Kästner et al. 2011], type checkers [Kästner et al. 2012], and model checkers [Classen et al. 2013]. Although the high level algorithm is the same as that of the corresponding single product analysis, re-developing each of these analyses from scratch is tedious, time consuming and potentially error prone. Other approaches tackle families of related analyses (e.g., data-flow analyses [Bodden et al. 2013] and abstract interpretation [Midtgaard et al. 2015]) instead of an individual analysis. This generalizes to more analyses, given they belong to the supported family.

A variability-aware analysis has to preserve the semantics of the original analysis, while applying it simultaneously to inputs from different product variants. At the same time, it should leverage the commonalities across the different variants as much as possible to minimize the computational overhead, and maximize the sharing of computed intermediate values. In theory the principles and techniques used in developing variability-aware analyses are the same regardless of the analysis itself. This raises the question of whether we can come up with a *generic* approach to *automatically* turn an *arbitrary* analysis program into its equivalent variability-aware analysis. This has been accomplished for Datalog-based analyses [Shahin and Chechik 2020; Shahin et al. 2019]. Yet, Datalog is less expressive than Turing-complete languages, and many useful analyses cannot be encoded as Datalog rules. For example, program analyses that compute monotone functions over lattices cannot be expressed in plain Datalog, and require functional programming extensions (e.g., Flix [Madsen et al. 2016], Datafun [Arntzenius and Krishnaswami 2016]).

In this paper, we address the problem of lifting arbitrary software analyses written in Turing-complete languages. We define a generic approach to automatically rewrite an analysis program G into program $G^\uparrow$, where $G^\uparrow$ takes an SPL as input. The output of $G^\uparrow$ is equivalent to the collective results of applying G to every product variant in the SPL. We refer to $G^\uparrow$ as the *lifted* version of G .

<pre> 1 int foo 2 3 #ifdef A // A 4 (int a) { 5 #ifdef B // A /\ B 6 return a * 2; 7 #else // A /\ !B 8 return a * 3 + 1; 9 #endif 10 11 #else // !A 12 (int a, int b) { 13 return (a 14 #ifdef B // !A /\ B 15 + 16 #else // !A /\ !B 17 * 2 + 18 #endif 19 b); 20 #endif 21 22 }</pre>	<pre> // A /\ B int foo (int a) { return a * 2; } // A /\ !B int foo (int a) { return a * 3 + 1; } // !A /\ B int foo (int a, int b) { return (a + b); } // !A /\ !B int foo (int a, int b) { return (a * 2 + b); }</pre>
---	--

Fig. 1. Example of an annotative product line, with the SPL on the left-hand side, and the four products that can be generated from it on the right-hand side.

Motivating Example. Fig. 1 shows a simple example of an SPL written in the C language, where C Pre-Processor (CPP) conditional compilation macros are used to associate different lines of code to different features. The code snippets within `#ifdef`-`#endif` blocks are only present when the `#ifdef` expression evaluates to True. By setting/unsetting different macros, those expressions can evaluate to True or False, and thus at preprocessing time we can vary the source code that actually gets compiled. When each of those macros corresponds to a program feature, the CPP becomes a tool for enabling/disabling individual features. In the example of Fig. 1, the features are *A* and *B*, and the four product variants obtained by different feature configurations are shown on the right.

Assume we have a simple software analysis tool `tokenCount` that, given a lexically tokenized program as input, returns the number of tokens. This is a straightforward counting algorithm in the case of single products. However, given an SPL, the analysis algorithm needs to simultaneously count the tokens belonging to different product variants. For the SPL in Fig. 1, the number of tokens is 13, 15, 18 or 20, depending on which product variant we are applying `tokenCount` to.

Our goal is to automatically rewrite `tokenCount` into `tokenCount†`, which returns a set of all four counts, each annotated with its corresponding feature configuration. Furthermore, we require that `tokenCount†` does an *efficient* computation of the counts by exploiting the shared parts of the SPL. For example, all four products of the SPL in Fig. 1 contain "int foo" as a prefix, and thus its tokens should be counted only once and used as part of producing all four counts.

We present two approaches to lifting, *shallow* and *deep*:

Shallow lifting takes analysis *G* as a black-box, and only wraps it in `G†` which takes variability-aware parameters and passes their combinations down to *G*. This approach is light-weight and in

many cases more efficient than brute-force analysis of each product variant individually. However, it does not leverage the opportunity to share common intermediate values across different input combinations throughout the analysis. *Deep lifting*, on the other hand, is a program rewriting technique that syntactically transforms a program into a semantically equivalent variability-aware one. We present rewrite rules for different syntactic constructs of the input language, and show that those rules together can be used to compositionally generate a semantically equivalent variability-aware program.

We define the rewriting algorithm for programs written in an extended version of Programming Computable Functions (PCF) [Plotkin 1977], which is essentially Typed Lambda Calculus (TLC) plus a fixed point operator. PCF is a concise, Turing-complete, functional programming formalism, and thus it is as computationally expressive as any high-level programming language. Several recent program rewriting and program analysis projects, for example [Aguirre et al. 2017; Kavvos et al. 2019; Keidel and Erdweg 2019; Schöpp 2017], use PCF (or variants of it) as a source language because of its expressiveness and its relatively small set of syntactic constructs.

Contributions and Organization. In this paper, we make the following contributions:

- (1) We define the correctness criteria for variability-aware functional programs.
- (2) We present a light-weight technique for lifting functional programs (shallow lifting).
- (3) We present a rewriting technique of programs written in an expanded variant of PCF, into semantically equivalent variability-aware programs (deep lifting).
- (4) We provide sketches of correctness proofs of lifted function application.
- (5) We outline the design of a Haskell implementation of both shallow and deep lifting.
- (6) We evaluate shallow and deep lifting against brute-force SPL analysis, using six program analyses applied to the BusyBox product line of command-line tools.

The rest of this paper is organized as follows. Sec. 2 provides background and definitions. Our shallow lifting framework is described in Sec. 3, followed by the deep lifting framework in Sec. 4. Correctness is then covered in Sec. 5. Sec. 6 outlines the implementation of both frameworks, and evaluation results are presented and discussed in Sec. 7. We provide some discussion points in Sec. 8, compare our approach to related work in Sec. 9, and conclude in Sec. 10.

2 BACKGROUND

We start by summarizing the basic Software Product Line and PCF concepts used in this paper.

2.1 Software Product Lines

Different variants of an SPL have different *features*. A *Feature* is an externally visible attribute of a software system. Examples include pieces of functionality, support for peripheral devices, and performance optimizations. We follow the *annotative* product line approach [Czarnecki and Antkiewicz 2005; Kästner and Apel 2008; Rubin and Chechik 2012], formally defined below.

Definition 2.1 (SPL). An SPL $\mathcal{L}$ is a tuple (F, Φ, D, ϕ) where: (1) F is the set of features s.t. an individual product can be derived from $\mathcal{L}$ via a *feature configuration* $\rho \subseteq F$. (2) $\Phi \in \text{Prop}(F)$ is a propositional formula over F defining the valid set of feature configurations. Φ is called a *Feature Model (FM)*. (3) D is a set of program elements, called the *domain model*. The whole set of program elements is sometimes referred to as the *100% representation* of the SPL. (4) $\phi : D \rightarrow \text{Prop}(F)$ is a total function mapping each program element to a proposition (*feature expression*) defined over the set of features F . $\phi(e)$ is called the *Presence Condition (PC)* of element e , i.e., the set of product configurations in which e is present.

```

1  int foo(int x, int y) {
2  #if defined(A) && defined(B) && defined(C) // A /\ B /\ C
3      return x + y;
4  #else // !(A /\ B /\ C)
5      return x - y;
6  #endif
7  }

```

Fig. 2. An example of a C-language source file with three features, but only two effective combinations.

For the example in Fig. 1, the feature set of the product line contains two features: **A** and **B**. In this example, domain model elements are the individual syntactic tokens of the program. `#ifdef` CPP directives are used to annotate code snippets with feature names. For example, we have *four* variants of function `foo`. Two variants of `foo` (with feature **A**) have one parameter each (lines 4-10). The other two variants (where feature **A** is absent) have two parameters each (lines 12-19).

In this example all feature combinations of **A** and **B** are allowed, so the feature model Φ is equal to the proposition `True` (i.e., no combinations are invalid). However, more realistic SPLs usually allow only a proper subset of feature combinations.

The implementation of each of the four variants of function `foo` depends on feature **B**. Nesting `#ifdef` directives semantically evaluates to the conjunction of individual features. Syntactically, tokens on line 6 have presence condition $(A \wedge B)$, while those on line 8 have $(A \wedge \neg B)$.

Definition 2.2 (Feature Configuration). A valid feature configuration ρ of a product line $\mathcal{L}$ is a subset of its features that satisfies Φ , i.e., Φ evaluates to `True` when each feature f of Φ is substituted by `True` when $f \in \rho$ and by `False` otherwise. The set of all valid configurations in $\mathcal{L}$ is denoted by $\text{Conf}(\mathcal{L})$.

In our example, $\text{Conf}(\mathcal{L}) = \{\{\}, \{\mathbf{A}\}, \{\mathbf{B}\}, \{\mathbf{A}, \mathbf{B}\}\}$.

Definition 2.3 (Product Derivation). A product M is *derived from* the product line $\mathcal{L}$ under the feature configuration ρ if M contains those and only those elements from the domain model whose presence conditions are satisfied by ρ . We denote M as $\mathcal{L}|_{\rho}$.

For example, the right-hand column of Fig. 1 has the four different products that can be derived from the the SPL on the left-hand side, each with a configuration from $\text{Conf}(\mathcal{L})$.

Definition 2.4 (Effective Combinations). The number of effective combinations of a product line $\mathcal{L}$ is the number of lexically unique products that can be derived from $\mathcal{L}$ under all valid feature combinations.

The number of products that can be derived from a product line is exponential in the number of product line features in the worst case. However, in many cases not all feature combinations are explicitly differentiated in product line artifacts. For example, the product line in Fig. 2 has three features, but the presence conditions (lines 2 and 4) only differentiate two sets of combinations. The presence condition $A \wedge B \wedge C$ on line 2 denotes the combination $\{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$, while the presence condition $\neg(A \wedge B \wedge C)$ on line 4 denotes the set of the remaining seven combinations, all of which are lexically identical. Thus, the number of effective combinations in this example is only two.

2.2 PCF and PCF+

Programming Computable Functions (PCF) [Plotkin 1977] is a simple language based on Typed Lambda Calculus, plus a fixed point operator for general recursion. PCF is Turing-complete, but

$T ::= \text{nat} \mid \text{bool} \mid (T, T) \mid [T] \mid T \rightarrow T$	(type constructors)
$\Gamma ::= \emptyset \mid \Gamma, x : T$	(typing contexts)
$\text{binop} ::= + \mid - \mid * \mid /$	(binary operators)
$t ::= x \mid \lambda v : T. t \mid \text{fix } t \mid t \ t$	(lambda terms)
$\quad \mid \underline{n} \mid \text{true} \mid \text{false}$	(numerals and boolean constants)
$\quad \mid t \ \text{binop} \ t \mid \text{iszero } t$	(mathematical expressions)
$\quad \mid \text{if } t \text{ then } t \text{ else } t$	(conditional)
$\quad \mid \text{case } t \text{ of } p \rightarrow t, \dots, p \rightarrow t$	(pattern matching)
$\quad \mid (t, t) \mid \text{fst}(t) \mid \text{snd}(t)$	(pair expressions)
$\quad \mid \text{nil} \mid \text{cons } t \ t \mid \text{isnil } t \mid \text{head } t \mid \text{tail } t$	(list expressions)
$p ::= v \mid \underline{n} \mid \text{true} \mid \text{false} \mid (p, p) \mid \text{cons } p \ p$	(patterns)

(a) Syntax

$\frac{}{\Gamma \vdash \underline{n} : \text{nat}}$	T-numeral	$\frac{}{\Gamma \vdash \text{true} : \text{bool}}$	T-true	$\frac{}{\Gamma \vdash \text{false} : \text{bool}}$	T-false
$\frac{x : T \in \Gamma}{\Gamma \vdash x : T}$	T-var	$\frac{\Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2}$	T-abs	$\frac{\Gamma \vdash t_1 : T_1 \rightarrow T_2 \quad \Gamma \vdash t_2 : T_1}{\Gamma \vdash t_1 \ t_2 : T_2}$	T-app
$\frac{\Gamma \vdash t : T \rightarrow T}{\Gamma \vdash \text{fix } t : T}$	T-fix	$\frac{\Gamma \vdash t_1 : \text{nat} \quad \Gamma \vdash t_2 : \text{nat}}{\Gamma \vdash t_1 \ \text{binop} \ t_2 : \text{nat}}$	T-binop		
$\frac{\Gamma \vdash t : \text{nat}}{\Gamma \vdash \text{iszero } t : \text{bool}}$	T-iszero	$\frac{\Gamma \vdash t_1 : \text{bool} \quad \Gamma \vdash t_2 : T \quad \Gamma \vdash t_3 : T}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}$	T-cond		
$\frac{\Gamma \vdash t : T_1 \quad \Gamma \vdash t_0 : T_2 \dots \Gamma \vdash t_n : T_2}{\Gamma \vdash \text{case } t \text{ of } p_0 \rightarrow t_0, \dots, p_n \rightarrow t_n : T_2}$	T-case	$\frac{\Gamma \vdash t_1 : T_1 \quad \Gamma \vdash t_2 : T_2}{\Gamma \vdash (t_1, t_2) : (T_1, T_2)}$	T-pair		
$\frac{\Gamma \vdash (t_1, t_2) : (T_1, T_2)}{\Gamma \vdash \text{fst}(t_1, t_2) : T_1}$	T-fst	$\frac{\Gamma \vdash (t_1, t_2) : (T_1, T_2)}{\Gamma \vdash \text{snd}(t_1, t_2) : T_2}$	T-snd		
$\frac{}{\Gamma \vdash \text{nil} : [T]}$	T-nil	$\frac{\Gamma \vdash t_1 : T \quad \Gamma \vdash t_2 : [T]}{\Gamma \vdash \text{cons } t_1 \ t_2 : [T]}$	T-cons	$\frac{\Gamma \vdash t : [T]}{\Gamma \vdash \text{isnil } t : \text{bool}}$	T-isnil
$\frac{\Gamma \vdash t : [T]}{\Gamma \vdash \text{head } t : T}$	T-head	$\frac{\Gamma \vdash t : [T]}{\Gamma \vdash \text{tail } t : [T]}$	T-tail		

(b) Typing rules.

Fig. 3. PCF+ syntax and typing rules.

still minimal in terms of constructs and data types. We begin by extending PCF to add polymorphic pairs and lists (to demonstrate lifting of product and sum types), and pattern matching using *case* expressions (along the lines of [Mitchell 1996; Pierce 2002]). We refer to the resulting language as PCF+. Fig. 3 summarizes the PCF+ syntax and typing rules. PCF+ Call By Name (CBN) semantics is summarized in Fig. 4.

Syntax. A syntactic term of PCF+ (Fig. 3a) is either a variable, a lambda abstraction, a recursive function definition (using a fixed point operator), lambda application, a numeral (underlined to

$$\begin{array}{c}
\frac{}{\underline{n} \downarrow n} \text{E-numeral} \quad \frac{M \downarrow m \quad N \downarrow n}{M \text{ binop } N \downarrow m \llbracket \text{binop} \rrbracket n} \text{E-binop} \\
\\
\frac{M \downarrow \lambda x.P \quad P[N/x] \downarrow V}{MN \downarrow V} \text{E-app} \\
\\
\frac{t_1 \downarrow t_2}{\text{fix } t_1 \downarrow \text{fix } t_2} \text{E-fix} \quad \frac{}{\text{fix}(\lambda x : T.t) \downarrow t[\text{fix}(\lambda x : T.t)/x]} \text{E-fixBeta} \\
\\
\frac{t \downarrow 0}{\text{iszero } t \downarrow \text{true}} \text{E-iszero1} \quad \frac{t \downarrow v \quad v \neq 0}{\text{iszero } t \downarrow \text{false}} \text{E-iszero2} \\
\\
\frac{B \downarrow \text{true} \quad M \downarrow v}{\text{if } B \text{ then } M \text{ else } N \downarrow v} \text{E-cond1} \quad \frac{B \downarrow \text{false} \quad N \downarrow v}{\text{if } B \text{ then } M \text{ else } N \downarrow v} \text{E-cond2} \\
\\
\frac{t \downarrow u \quad \text{match}(p_i, u) \wedge \forall_{j < i} : \neg \text{match}(p_j, u) \quad t_i[v_0/x_0, \dots, v_k/x_k] \downarrow w}{\text{case } t \text{ of } p_0 \rightarrow t_0, \dots, p_n \rightarrow t_n \downarrow w} \text{E-case} \\
\\
\frac{M \downarrow \text{nil}}{\text{isnil } M \downarrow \text{true}} \text{E-isnil1} \quad \frac{M \downarrow \text{cons } N \ P}{\text{isnil } M \downarrow \text{false}} \text{E-isnil2} \\
\\
\frac{M \downarrow v}{\text{fst } (M, N) \downarrow v} \text{E-fst} \quad \frac{N \downarrow v}{\text{fst } (M, N) \downarrow v} \text{E-snd} \quad \frac{M \downarrow v}{\text{head } (\text{cons } M \ N) \downarrow v} \text{E-head} \\
\\
\frac{}{\text{tail nil} \downarrow \text{nil}} \text{E-tail1} \quad \frac{N \downarrow v}{\text{tail } (\text{cons } M \ N) \downarrow v} \text{E-tail2}
\end{array}$$

Fig. 4. Call-By-Name (CBN) Operational semantics of PCF+.

distinguish it from its semantic natural number), a mathematical expression (using addition, subtraction, multiplication and division operators), a Boolean constant (`true` or `false`), a conditional expression, a `case` expression (pattern matching), a constructor expression of a pair, or a constructor expression of a list. The language includes primitives for pair and list manipulation (`fst`, `snd`, `isnil`, `head`, and `tail`). Each well-typed syntactic term has a unique type that is either `nat`, `bool`, (T_1, T_2) , $[T]$, or a unary function type $(T_1 \rightarrow T_2)$ from type T_1 to type T_2 . The language supports functions of higher arities using currying.

Typing rules. Given a global typing context Γ , each well-typed term is assigned a type according to the typing rules of Fig. 3b. All numeral constants have type `nat`, and Boolean constants `true` and `false` have type `bool`. Variable types are stored in the typing context Γ . A lambda abstraction from type T_1 to type T_2 has type $T_1 \rightarrow T_2$. A lambda application term $t_1 t_2$ has type T_2 if t_1 has type $T_1 \rightarrow T_2$ and t_2 has type T_1 . A fixed point over a function of type $T \rightarrow T$ has type T (the type system does not designate a type for diverging functions).

Applying an infix binary operator to two terms of type `nat` is a term of type `nat` (again, exceptions like division by zero are not designated a type). Applying the `iszero` unary operator to a term of type `nat` results in a term of type `bool`. An if-then-else term is of type T if both the then-term


```

1 tokenCount = fix  $\lambda x : [\text{nat}]$  .
2               case  $x$  of
3                 nil  $\rightarrow \underline{0}$ 
4                 cons  $h\ t \rightarrow$ 
5                     if iszero  $h$ 
5                     then tokenCount  $t$ 
6                     else  $\underline{1} + \text{tokenCount } t$ 

```

Fig. 5. tokenCount analysis.

and else-term are of type T , and the condition is of type bool . A case expression has type T_2 if each of its alternatives is an expression of type T_2 .

The pair type has one constructor, taking two arguments of types T_1 and T_2 , respectively, and constructing a value of type (T_1, T_2) . A list over type T has two constructors: `nil` constructing an empty list, and `cons` constructing a list from a value of type T and a $[T]$. The `isnil` unary operator takes a $[T]$ and returns a bool . Operators `head` and `tail` each takes a $[T]$ and returns a T and a $[T]$, respectively.

Operational semantics. Fig. 4 outlines the *Call-By-Name (CBN)* operational semantics of PCF. Constants (`true`, `false`, and `nil`) are irreducible normal forms. A numeral $\underline{n}$ reduces to its corresponding natural number. A binary mathematical operator symbol *binop* reduces to its corresponding mathematical operator $\llbracket \text{binop} \rrbracket$, so when it is used as an infix operator on two terms reducing to natural numbers m and n , it reduces to the result of the mathematical expression $m \llbracket \text{binop} \rrbracket n$.

Function application substitutes the argument in place of the parameter in the function body. A fixed point operator applies its argument to itself (allowing for the definition of recursive functions). The `iszero` operator returns `true` if its argument reduces to zero, and returns `false` otherwise. An if-then-else term reduces to what the then-term reduces to if the condition reduces to `true`, and reduces to what the else-term reduces to otherwise.

A case expression evaluates its argument t to value u , and tries to match the structure of u against each of the patterns one-by-one (in order). The alternative expression t_i of the first matching pattern p_i is then reduced to value w , substituting each of the pattern variables x with its corresponding matching value v from u . The value w is what the overall case expression reduces to.

Expressions $(\text{fst } M)$ and $(\text{snd } M)$ reduce to values x and y , respectively, if M reduces to (x, y) . Operator `isnil` applied to a term M reduces to `true` if M reduces to `nil`, and reduces to `false` otherwise. Operator `head` applied to a term $(\text{cons } M\ N)$ reduces to v if M reduces to v . Operator `tail` reduces to `nil` if applied to `nil`, and to v if applied to $(\text{cons } M\ N)$ and N reduces to v .

PCF is Turing-complete [Plotkin 1977], allowing for the definition of partial functions, so we assume undefined behavior (e.g., division by zero, head of an empty list) to be semantically equivalent to diverging expressions.

3 SHALLOW LIFTING

Consider the token counting example from Fig. 1. Assume each token has a unique non-zero natural number identifier, where identifier $\underline{0}$ corresponds to a special empty token. Fig. 5 shows a PCF+ implementation of the tokenCount analysis introduced in Sec. 1. This analysis cannot take an SPL (like the one in Fig. 1) as input because it cannot handle variability. A variability-aware version of tokenCount (let us call it tokenCount¹) needs to take a variability-aware list as input,

return a variability-aware result (token counts for each of the program variants), and in order to compute the correct results, it has to systematically track intermediate values and associate each of them to its corresponding variant. This all has to be done while maintaining the semantics of the original tokenCount analysis.

Our goal is to automatically lift an arbitrary PCF+ program G to its semantically equivalent lifted program $G^\uparrow$. In this section, we achieve that goal by taking G as a black-box, and wrapping it in $G^\uparrow$. We call this approach *shallow lifting*.

3.1 Variability-Aware Values

In PCF+, a value v of type T is a singleton belonging to that type. For example, $\underline{7}$ is a value of type nat . In variability-aware domains, on the other hand, a variable of type T can simultaneously have different values of type T , each in a different set of products identified by a *Presence Condition* (PC).

For example, the SPL in Fig. 1 has two features, A and B , and PCs are Boolean expressions over those two features. The product space is partitioned into four subsets labeled by the PCs $(A \wedge B)$, $(A \wedge \neg B)$, $(\neg A \wedge B)$, and $(\neg A \wedge \neg B)$. The result of our token count analysis differs from one set of products to another. For the example in Fig. 1, the number of tokens is 13 in $(A \wedge B)$, 15 in $(A \wedge \neg B)$, 18 in $(\neg A \wedge B)$, and 17 in $(\neg A \wedge \neg B)$. That variability-aware value is the set of nat -PC pairs $\{(\underline{13}, A \wedge B), (\underline{15}, A \wedge \neg B), (\underline{18}, \neg A \wedge B), (\underline{17}, \neg A \wedge \neg B)\}$. The first element of each pair is a nat value (we call that an *atomic value*), and the second element is a set of products denoted by a presence condition.

We assume that the PC type encapsulates presence conditions over the feature space. A variability-aware value over a type T is a set of pairs (v, pc) , where v is of type T and pc is of type PC, and the following *disjointness* and *full coverage* invariants hold.

DISJOINTNESS INVARIANT. Given a variability-aware value $v^\uparrow$:

$$v^\uparrow = \{(v_1, pc_1), \dots, (v_n, pc_n)\}, \forall i \neq j : \text{unsat}(pc_i \wedge pc_j) \quad (1)$$

The disjointness invariant intuitively states that a variability-aware value can have at most one atom value in any product configuration. The conjunction of two presence conditions is unsatisfiable if and only if they represent non-overlapping sets of products. Without this invariant, the semantics of variability-aware values would be non-deterministic. This invariant is a precondition on program inputs, and in Sec. 5 we show how its validity is maintained by the lifting transformations.

The second invariant states that a variability-aware value $v^\uparrow$ covers the full product space, not leaving out any valid feature combination.

FULL COVERAGE INVARIANT.

$$v^\uparrow = \{(v_1, pc_1), \dots, (v_n, pc_n)\}, \bigvee_i pc_i = \Phi \quad (2)$$

The two invariants together ensure that, semantically, a variability-aware value $v^\uparrow$ of type T is a total mapping from the set of valid product configurations to the underlying data type T :

$$\llbracket v^\uparrow : T^\uparrow \rrbracket : \Phi \rightarrow T$$

Definition 3.1 (Lifted Value Indexing). Given a lifted value $v^\uparrow$ and a configuration ρ , $v^\uparrow|_\rho$ is the atomic value of $v^\uparrow$ whose presence conditions is satisfied by ρ .

Example 3.2 (Lifted Variables). Given features A and B , variables x , y , and z of type $\text{nat}^\uparrow$ can have the following values:

$$\begin{aligned} x &= \{(\underline{1}, A), (\underline{2}, \neg A \wedge B), (\underline{0}, \neg A \wedge \neg B)\} \\ y &= \{(\underline{5}, A \wedge \neg B), (\underline{4}, B), (\underline{3}, \neg A \wedge \neg B)\} \end{aligned}$$

$z = \{(19, \text{True})\}$

Variable x has the value $\underline{1}$ when feature A is present. If A is absent, x is $\underline{2}$ if A is present, and $\underline{0}$ if B is absent. Variable y has the value $\underline{4}$ if B is present, $\underline{5}$ if B is absent and A is present, and $\underline{3}$ if both features are absent. Variable z has the value $\underline{19}$ in all products regardless of which features are present or absent.

Examples of applying the value indexing operator are $x|_{\{A\}} = \underline{1}$ (A is present and B is absent), $y|_{\{A,B\}} = \underline{4}$ (both A and B are present), and $z|_{\{\}} = \underline{19}$ (none of the two features is present).

3.2 Variability-Aware Types

Given a type T , its corresponding variability-aware type $T^\uparrow$ is the type of sets of T-PC pairs satisfying both the disjointness and completeness invariants:

$$\frac{v_1, \dots, v_n : T, pc_1, \dots, pc_n : PC \quad \forall i \neq j. \text{unsat}(pc_i \wedge pc_j) \quad \bigvee_i pc_i = \Phi}{\{(v_1, pc_1), \dots, (v_n, pc_n)\} : T^\uparrow} \text{lifted-type}$$

Note that we are only restricting input programs to be in PCF+, but we can rewrite them into a language with a richer type system with polymorphic types and type classes (e.g., Haskell). In Sec. 6, we show how lifted types can be implemented as a Haskell polymorphic type.

3.3 Lifting Function Application

In PCF+ (and functional programming in general), functions are values. Following our treatment of lifted values, a function f of type $(a \rightarrow b)$ is lifted to $f^\uparrow$ of type $(a^\uparrow \rightarrow b^\uparrow)$. However, following the PCF+ type system rules, $f^\uparrow$ should be of type $a^\uparrow \rightarrow b^\uparrow$ if we are to apply it to values of type $a^\uparrow$ and get values of type $b^\uparrow$ back. As a result, we need to introduce our own lifted function application mechanism on top of PCF+ function application.

Starting with the simplest case, where we have a lifted unary function of type $(a \rightarrow b)$ and an argument of type $a^\uparrow$, we need to provide a way to apply that function to its argument resulting in a $b^\uparrow$ value. Recall that a lifted value of type $t^\uparrow$ is just a set of values of type t , each paired with a PC (with the whole set satisfying the disjointness and full coverage invariants). To apply a collection of functions to a collection of values, we only need to apply each of the functions to each of the values. Since the result has to be a lifted value, we also need a PC for each of the computed values. Intuitively, since the result exists only if its generating function and argument exist, then its PC is the conjunction of the PCs of the function and the argument. If the conjunction of the PCs is unsatisfiable (i.e., a logical contradiction), it means that the set of products where the function is defined and that where the atomic value is defined do not intersect. Consequently, the result can be safely filtered out.

The apply operator does exactly that. It generates the cross product of functions and arguments from f and x . For each element in the cross product (a pair of pairs), it conjoins the individual PCs, and filters the element out if the conjunction is not satisfiable. The semantics of apply is summarized by the following inference rule:

$$\frac{f : (a \rightarrow b)^\uparrow \quad x : a^\uparrow}{(\text{apply } f \ x : b^\uparrow) = \{(f'(x'), fpc \wedge xpc) \mid (f', fpc) \in f, (x', xpc) \in x, \text{sat}(fpc \wedge xpc)\}} \text{apply}$$

Example 3.3 (Unary function application). Assume we need to apply the `iszero` function to the lifted variable x from the Example 3.2. First, we need a lifted version of `iszero`. Since the behavior of `iszero` does not vary across different product configurations, we can just wrap it into a singleton lifted value with presence condition `True`: $\text{iszero}^\uparrow = \{(\text{iszero}, \text{True})\}$.

Applying $\text{iszero}^\uparrow$ to x evaluates to:

$$\begin{aligned}
\text{apply iszero}^\uparrow x &= \{ (\text{iszero } \underline{1}, \text{True} \wedge A), \\
&\quad (\text{iszero } \underline{2}, \text{True} \wedge \neg A \wedge B), \\
&\quad (\text{iszero } \underline{0}, \text{True} \wedge \neg A \wedge \neg B) \} \\
&= \{ (\text{false}, A), \\
&\quad (\text{false}, \neg A \wedge B), \\
&\quad (\text{true}, \neg A \wedge \neg B) \}
\end{aligned}$$

Example 3.4 (Binary function application). Now assume that we would like to add x and y from Example 3.2. The first step is to get a lifted version of the $(+)$ binary operator: $(+)^\uparrow = \{((+), \text{True})\}$.

We cannot directly use `apply` because it expects only two arguments, the first of which is a lifted unary function. However, functions in PCF are all unary, and functions of higher arities are just Curried higher-order functions [Mitchell 1996]. If we pass $(+)^\uparrow$ and x to `apply`, it will apply the binary lifted operator $(+)^\uparrow$ to a single lifted argument x :

$$(i : (\text{nat} \rightarrow \text{nat})^\uparrow) = \text{apply } (+)^\uparrow x$$

Since multi-parameter functions are Curried unary functions, the intermediate value i is a lifted unary function value of type $(\text{nat} \rightarrow \text{nat})^\uparrow$, evaluating to:

$$\begin{aligned}
i = \text{apply } (+)^\uparrow x &= \{ ((+)^\uparrow \underline{1}, \text{True} \wedge A), \\
&\quad ((+)^\uparrow \underline{2}, \text{True} \wedge \neg A \wedge B), \\
&\quad ((+)^\uparrow \underline{0}, \text{True} \wedge \neg A \wedge \neg B) \} \\
&= \{ ((\underline{1}+), A), \\
&\quad ((\underline{2}+), \neg A \wedge B), \\
&\quad ((\underline{0}+), \neg A \wedge \neg B) \}
\end{aligned}$$

Now we just need to apply i , which is a lifted unary function of type $(\text{nat} \rightarrow \text{nat})^\uparrow$, to y :

$$\begin{aligned}
\text{apply } i \ y &= \{ ((\underline{1}+) \ \underline{5}, A \wedge A \wedge \neg B), \\
&\quad ((\underline{1}+) \ \underline{4}, A \wedge B), \\
&\quad ((\underline{1}+) \ \underline{3}, A \wedge \neg A \wedge \neg B), \\
&\quad ((\underline{2}+) \ \underline{5}, \neg A \wedge B \wedge A \wedge \neg B), \\
&\quad ((\underline{2}+) \ \underline{4}, \neg A \wedge B \wedge B), \\
&\quad ((\underline{2}+) \ \underline{3}, \neg A \wedge B \wedge \neg A \wedge \neg B), \\
&\quad ((\underline{0}+) \ \underline{5}, \neg A \wedge \neg B \wedge A \wedge \neg B), \\
&\quad ((\underline{0}+) \ \underline{4}, \neg A \wedge \neg B \wedge B), \\
&\quad ((\underline{0}+) \ \underline{3}, \neg A \wedge \neg B \wedge \neg A \wedge \neg B), \\
&= \{ (\underline{6}, A \wedge \neg B), \\
&\quad (\underline{5}, A \wedge B), \\
&\quad (\underline{4}, \text{False}), \\
&\quad (\underline{7}, \text{False}), \\
&\quad (\underline{6}, \neg A \wedge B), \\
&\quad (\underline{5}, \text{False}), \\
&\quad (\underline{5}, \text{False}), \\
&\quad (\underline{4}, \text{False}), \\
&\quad (\underline{3}, \neg A \wedge \neg B) \}
\end{aligned}$$

Eliminating the pairs with `False` (unsatisfiable) presence conditions, we are left with exactly four pairs, one for each product configuration.

Versions of `apply` of different arities can thus be defined using straightforward Currying. For example, the implementation of `apply2` of type $(a \rightarrow b \rightarrow c)^\uparrow \rightarrow a^\uparrow \rightarrow b^\uparrow \rightarrow c^\uparrow$ is

$$\text{apply2 } f \ x \ y = \text{apply } (\text{apply } f \ x) \ y.$$

foo = $\lambda x : \text{nat}. \lambda y : \text{nat}. \lambda z : \text{nat}.$
 (bar x y) + (baz z)

(a) Function foo.

x = $\{(-7, A), (3, \neg A)\}$
 y = $\{(1, A \wedge B), (8, A \wedge \neg B), (4, \neg A \wedge B), (10, \neg A \wedge \neg B)\}$
 z = $\{(5, \text{True})\}$

(b) Arguments x, y, and z.

foo[↑] = $\{(\text{foo}, \text{True})\}$
 result = apply3 foo[↑] x y z

(c) Shallow-lifted foo[↑] applied to x, y, and z.

x	y	z	PC
-7	1	5	$A \wedge B$
-7	8	5	$A \wedge \neg B$
-7	4	5	False
-7	10	5	False
3	1	5	False
3	8	5	False
3	4	5	$\neg A \wedge B$
3	10	5	$\neg A \wedge \neg B$

(d) Input vectors for foo[↑].

Fig. 6. Shallow lifting foo into foo[↑], and applying foo[↑] to x, y, and z. Fig.(d) lists the input vectors passed to foo, with those with unsatisfiable PCs crossed out.

Shallow lifting wraps a function in its lifted counterpart. While straightforward, it treats the original function as a black-box, so it does not leverage opportunities for reuse of common intermediate values within that function. For example, function foo in Fig. 6a takes three nat arguments, passing the first two to function bar and the third to function baz. If we are to call the shallow-lifted function foo[↑] (Fig. 6c) with arguments in Fig. 6b, the original function foo gets called with the input vectors in Fig. 6d (those with unsatisfiable PCs are crossed out).

The column for argument z has the value 5 for all input vectors. As a result, foo will internally call baz four times, each with the exact same argument 5. This is an example of a case where an intermediate value should be computed only once and subsequently reused. Since shallow lifting does not introspect the structure of the function being lifted, those value sharing opportunities are wasted.

4 DEEP LIFTING

Deep lifting rewrites the body of a function, pushing lifted function application down into the function body. For example, a deep-lifted foo (from Fig. 6) would look like this:

foo[↑] = $\lambda x : \text{nat}^\uparrow. \lambda y : \text{nat}^\uparrow. \lambda z : \text{nat}^\uparrow.$
 apply2 (+)[↑] (apply2 bar[↑] x y) (apply baz[↑] z)

Assuming bar[↑] and baz[↑] are shallow-lifted versions of bar and baz respectively, redundant calls to them will be eliminated. For example, since z is a singleton, applying baz[↑] to it would result in only one call to the underlying function baz instead of four redundant calls as in the case of shallow-lifted foo.

In this section, we present the rewrite rules for the syntactic constructs of PCF+ that require recursive rewriting (primitive constructs are shallow-lifted). The $\Rightarrow$ symbol stands for "rewrite-into".

4.1 Lifting Function Definitions

A function body is a PCF+ term. Primitive values (e.g., numerals, Boolean constants) and calls to primitive built-in functions (e.g., mathematical operators) are shallow lifted because they do not have an internal structure to push lifting into. Calls to user-defined functions (e.g., bar and baz in Fig. 6) can be either shallow-lifted or deep-lifted. If the called function belongs to another module that is not being deep-lifted, then the function call is shallow-lifted using the different variants of the apply operator. If on the other hand the callee belongs to the same module or to another module

that is being deep-lifted then the call is left as is. The deep-lifted callee expects variability-aware arguments, and the parameters passed by the deep-lifted caller are variability-aware, so built-in PCF+ function application can be used right away.

4.2 Lifting Conditional Expressions

In Call-By-Name(CBN) functional programming, a conditional expression is just syntactic sugar for a ternary polymorphic function, taking a Boolean argument and two arguments of type T and returning one of them depending on what the first argument evaluates to. Only one branch of the conditional expression is evaluated in CBN, based on what the boolean expression evaluates to. Thus it is tempting to treat conditional expressions as functions, and use lifted function application to lift them. However, conditional expressions in a variability-aware setting are more subtle.

For example, assume we are lifting the following expression (with values from Example 3.2):

$\text{if } (\text{iszero } x) \text{ then } (x + z) \text{ else } (z / x)$

If we are to treat this expression as a ternary function and apply its lifted version to the three arguments using Currying, we would violate the semantics of the original expression and throw a division-by-zero exception. The reason is that although CBN (lazy evaluation) evaluates expressions only when needed, both $(x + z)$ and (z / x) actually need to be evaluated because $\text{iszero } x$ evaluates to true in configuration $\neg A \wedge \neg B$ and to false in the rest of the configurations. The results of the addition and division mathematical expressions will then be filtered based on the true and false configurations. In the general case, this is not optimal because some computation results will be thrown away, but even worse, in this particular case, it also results in a faulty behavior violating the intended semantics (division-by-zero).

To preserve the semantics of conditional expressions, we need to filter the expressions of the *then* and *else* branches *before* they are evaluated as follows:

$$\frac{t_1^\uparrow \downarrow \{(\text{true}, x), (\text{false}, y)\}}{(\text{if } t_1 \text{ then } t_2 \text{ else } t_3)^\uparrow \downarrow \llbracket t_2^\uparrow \rrbracket_x \cup \llbracket t_3^\uparrow \rrbracket_y} \text{ lifted-if}$$

Example 4.1 (Conditional Expressions). According to the lifted-if inference rule, this expression evaluates as follows:

$$\begin{aligned} & (\text{if } (\text{iszero } x) \text{ then } (x + z) \text{ else } (z / x))^\uparrow \\ &= \text{if}^\uparrow \{(\text{iszero } \underline{1}, A), (\text{iszero } \underline{2}, \neg A \wedge B), (\text{iszero } \underline{0}, \neg A \wedge \neg B)\} \\ & \quad \text{then}^\uparrow (x(+)^{\uparrow} z) \text{ else}^\uparrow (z (/)^{\uparrow} x) \\ &= \text{if}^\uparrow \{(\text{false}, A \vee (\neg A \wedge B)), (\text{true}, \neg A \wedge \neg B)\} \text{ then}^\uparrow (x(+)^{\uparrow} z) \text{ else}^\uparrow (z (/)^{\uparrow} x) \\ &= (x(+)^{\uparrow} z)_{\neg A \wedge \neg B} \cup (z (/)^{\uparrow} x)_{A \vee (\neg A \wedge B)} \\ &= (\{\underline{0}, \neg A \wedge \neg B\} (+)^{\uparrow} \{\underline{19}, \neg A \wedge \neg B\}) \cup (\{\underline{19}, A \vee (\neg A \wedge B)\} (/)^{\uparrow} \{\underline{1}, A\}, \{\underline{2}, \neg A \wedge B\}) \\ &= \{\underline{19}, \neg A \wedge \neg B\} \cup \{\underline{19}, A\}, \{\underline{9}, \neg A \wedge B\} \\ &= \{\underline{19}, \neg A \wedge \neg B\}, \{\underline{19}, A\}, \{\underline{9}, \neg A \wedge B\} \end{aligned}$$

Operationally, to rewrite an arbitrary conditional expression we need two auxiliary rewrite operations: *lifted expression partitioning*, and *recursive expression restriction*. Given a lifted expression e of type $T^\uparrow$, a predicate p of type $T \rightarrow \text{bool}$, the *partition* operator fully evaluates e to lifted value v , then returns a pair of presence conditions (pc_1, pc_2) , where pc_1 is the disjunction of presence conditions of atomic values of v satisfying the predicate p , and pc_2 is the disjunction of presence conditions of the other atomic values.

The *restrict* rewriting operator takes a syntactic expression e and returns a syntactic lambda expression taking a presence condition pc as argument, and a body of e where each subterm t is

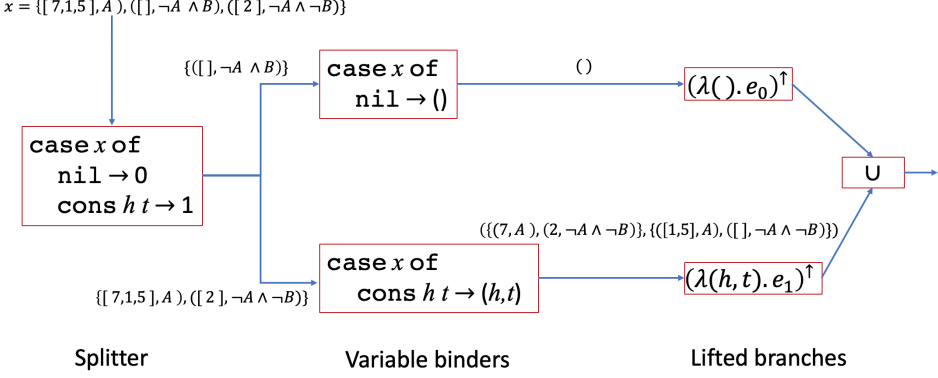


Fig. 7. The pipeline of the lifted case expression, applied to an example of a lifted list of integers.

replaced with $t|_{pc}$. Restricting all the terms of an expression to a set of products results in restricting the whole expression to that set. A conditional expression is then deeply lifted as follows:

$$(\text{if } c \text{ then } t_1 \text{ then } t_2)^\uparrow \Rightarrow (pc_1, pc_2) = \text{partition}(c, \lambda b.b) \\ ((\text{restrict } t_1) pc_1) \cup ((\text{restrict } t_2) pc_2)$$

4.3 Lifting Pattern Matching

Pattern matching generalizes conditional expressions to an arbitrary number of potentially overlapping conditions, each with a branching alternative. Moreover, a pattern might include variables that are bound to values as a part of the matching process. For example, the PCF+ program in Fig. 5 matches the list expression x against two patterns: `nil` for empty lists, and `cons h t` for non-empty ones. The second pattern binds the variables h to the head of x , and t to its tail.

In addition to splitting the lifted input across the different branching alternatives based on the matched patterns, lifted pattern matching needs to address two additional aspects: overlapping conditions and pattern variable binding. Since the conditions in a `case` expression might overlap (i.e., more than one can evaluate to true), a priority mechanism is usually specified as a part of the semantics of the language. In PCF+ (see Fig. 4), patterns are evaluated in order, and the branching alternative of the first matching pattern is the one evaluated. We need to preserve the same semantics when lifting a `case` expression. In addition, variables of the matching pattern need to be bound to values from the expression we are matching against. Those variables are in the scope of the corresponding branch expression, which is to be deep-lifted.

We lift `case` expressions into `case†` using a three-stage pipeline: splitting, binding and evaluation. This pipeline is demonstrated in Fig. 7 on an example of matching against a list expression. The first stage (the splitter) is a `case` expression mapping each element of the lifted input to the index of the matched pattern (0,1,...). The output of the first stage is a partition of the input lifted value. In the example, the input is partitioned into empty lists (matching the first pattern), and non-empty ones (matching the second). The first stage generates the following partitioning expression:

$$\text{case } x \text{ of } p_0 \rightarrow t_0, \dots, p_n \rightarrow t_n^\uparrow \Rightarrow \text{case } x \text{ of } p_0 \rightarrow 0, \dots, p_n \rightarrow n$$

The second stage binds pattern variables to tuples of lifted values. The first pattern in the example does not bind any variables, so the output of the first binder is an empty tuple. The second

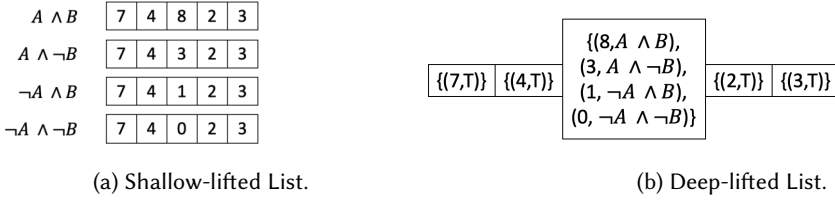


Fig. 8. Difference between (a) deep-lifted and (b) shallow-lifted lists.

pattern binds variable h to $(7, A)$, $(2, \neg A \wedge \neg B)$, and binds t to $([1, 5], A)$, $([], \neg A \wedge \neg B)$. Those are respectively the heads and tails of the matched elements of the input lifted value. The second stage generates the following variable binding expressions (where $\text{vars}(p_i)$ is the syntactic tuple $(v_{i,0}, \dots, v_{i,k})$ of variables in pattern p):

$$\begin{aligned} \text{case } x \text{ of } p_0 \rightarrow t_0, \dots, p_n \rightarrow t_n^\uparrow &\Rightarrow \text{case } x \text{ of } p_0 \rightarrow \text{vars}(p_0) \\ &\dots \\ &\text{case } x \text{ of } p_n \rightarrow \text{vars}(p_n) \end{aligned}$$

The third stage is the branches of the original case expression, each transformed into a lifted body of a lambda taking the tuple of bound values from stage two as input. The indices calculated in stage one are used to map atomic values to their matched lifted branches. The outputs of stage three are combined into a single lifted value, which is a straightforward set union operation. Each of the t_k branching expressions of the case expression is rewritten as follows:

$$t_k^\uparrow \Rightarrow \lambda(\text{vars}(p_k)).\text{restrict}(t_k)$$

Recall that *restrict* generates a lambda expression taking a presence condition as argument. The outputs from stage two together with presence conditions are passed to each of the expressions generated by stage three, and the union of the results is the result of the lifted case expression.

4.4 Deep Data

Monomorphic data types are lifted into sets of value-presence condition pairs. However, polymorphic data types (e.g., lists and pairs) carry values at multiple levels. For example, `[nat]` is a data structure at the outer level, carrying natural number values at the inner level. A shallow-lifted list (Fig. 8a) attaches a presence condition to the list data structure, keeping the inner values with their original type (`nat` in this example). A deep-lifted list on the other hand (Fig. 8b) keeps the original data structure as is, while lifting the inner values.

Under a deep-lifting framework, polymorphic data types (list and pairs) are lifted as follows:

$$\begin{aligned} [T]^\uparrow &\Rightarrow [T^\uparrow] \\ (T_1, T_2)^\uparrow &\Rightarrow (T_1^\uparrow, T_2^\uparrow) \end{aligned}$$

With the exception of `isnil`[†], using the same implementation of their unlifted counterparts, lifted list and pair operations have the following type signatures:

$$\begin{aligned} \text{cons}^\uparrow : T^\uparrow \rightarrow [T^\uparrow] \rightarrow [T^\uparrow] \\ \text{head}^\uparrow : [T^\uparrow] \rightarrow T^\uparrow \end{aligned}$$

$$\text{tail}^\uparrow : [T^\uparrow] \rightarrow [T^\uparrow]$$

$$\text{isnil}^\uparrow : [T^\uparrow] \rightarrow \text{bool}^\uparrow$$

$$\text{fst}^\uparrow : (T_1^\uparrow, T_2^\uparrow) \rightarrow T_1^\uparrow$$

$$\text{snd}^\uparrow : (T_1^\uparrow, T_2^\uparrow) \rightarrow T_2^\uparrow$$

Because $\text{isnil}^\uparrow$ is expected to return a lifted Boolean, we cannot use its unlifted version as is. Instead, it needs to pair the result of calling isnil with True as a presence condition (because of the total coverage invariant) as follows:

$$\text{isnil}^\uparrow x = \{(\text{isnil } x, \text{True})\}$$

Deep lifting of data structures has the advantage of minimizing data redundancy, and maximizing sharing. The example in Fig. 8 shows a variability-aware list of five elements, where the third element varies across different product variants, while all the other elements are the same across all variants. A shallow-lifting would result in four lists with four out of five positions carrying the same value. Even if the language compiler and run-time system are smart enough to share the memory where common elements are stored, when the shallow-lifted list is traversed the common elements will still be read multiple times.

The disadvantage of deep lifting of data structures on the other hand is that they need special case handling of their type signatures. The lifting framework assumes that any type T is lifted to type $T^\uparrow$. For example, a lifted list of natural numbers is expected to have the $[\text{nat}]^\uparrow$ type. A deep-lifted list of naturals would have the type $[\text{nat}^\uparrow]$ instead. In order to support deep lifting of data structures, we treat polymorphic types differently during rewriting. This requires maintaining type information of terms and expressions during the rewriting process.

5 CORRECTNESS OF LIFTED FUNCTION APPLICATION

Deep rewriting rules preserve the semantics of their respective language constructs by design. The underlying primitives of the lifting framework are lifted values (with their invariants) and lifted function application. In this section, we specify the correctness criteria of lifted programs, and prove that lifted function application preserves the disjointness and total coverage invariants of lifted values, and the correctness criteria of lifting.

Definition 5.1 (Correctness of Lifting). Given a product line $\mathcal{L}$, a PCF+ function f and a configuration ρ , $f^\uparrow$ is a correct lifting of f if and only if indexing $f^\uparrow$ applied to $\mathcal{L}$ by ρ is equal to the result of applying f to the product generated from $\mathcal{L}$ using configuration ρ . Formally, $f^\uparrow(\mathcal{L})|_\rho = f(\mathcal{L}|_\rho)$.

This is summarized by the following commuting diagram:

$$\begin{array}{ccc} \mathcal{L} & \xrightarrow{f^\uparrow} & R^\uparrow \\ \downarrow |_\rho & & \downarrow |_\rho \\ \mathcal{L}|_\rho & \xrightarrow{f} & R \end{array}$$

5.1 Application Preserves Disjointness

THEOREM 5.2. *If both $f^\uparrow$ and $x^\uparrow$ satisfy the disjointness invariant, then the result of $(\text{apply } f^\uparrow x^\uparrow)$ also satisfies disjointness.*

PROOF. By contradiction:

Assume that both $f^\uparrow$ and $x^\uparrow$ satisfy the disjointness invariant while the result $y = (\text{apply } f^\uparrow x^\uparrow)$ does not. Then there exists at least one pair of presence conditions in y (pc_i and pc_j) where $\text{sat}(pc_i \wedge pc_j) = \text{True}$.

Each of pc_i and pc_j is a conjunction of presence conditions from $f^\uparrow$ and $x^\uparrow$ (the apply inference rule), then for arbitrary a, b, c and d :

$$\begin{aligned} pc_i &= fpc_a \wedge xpc_b \\ pc_j &= fpc_c \wedge xpc_d \end{aligned}$$

Then $pc_i \wedge pc_j = fpc_a \wedge xpc_b \wedge fpc_c \wedge xpc_d$.

For this conjunction to be satisfiable, all pair-wise conjunctions of its terms need to be satisfiable. This includes both $\text{sat}(fpc_a \wedge fpc_c)$ and $\text{sat}(xpc_b \wedge xpc_d)$, which contradicts the assumption that $f^\uparrow$ and $x^\uparrow$ both satisfy disjointness. Then $y = (\text{apply } f^\uparrow x^\uparrow)$ must satisfy disjointness too. $\square$

5.2 Application Preserves Full Coverage

THEOREM 5.3. *If both $f^\uparrow$ and $x^\uparrow$ satisfy the full coverage invariant, then the result of $(\text{apply } f^\uparrow x^\uparrow)$ also satisfies full coverage.*

PROOF. Given an arbitrary product configuration $\rho \in \Phi$, there exists $(f', fpc) \in f^\uparrow$ and $(x', xpc) \in x^\uparrow$ where $f^\uparrow|_\rho = f'$ and $x^\uparrow|_\rho = x'$ (since both $f^\uparrow$ and $x^\uparrow$ satisfy the full-coverage invariant).

Then $(f' x', fpc \wedge xpc) \in (\text{apply } f^\uparrow x^\uparrow)$ (apply inference rule). Since ρ is an arbitrary configuration, then each configuration will be covered in the result of lifted application (i.e., lifted application preserves full coverage). $\square$

5.3 Application Preserves Correctness

THEOREM 5.4. *If $f^\uparrow$ is a lifting of function f , and $x^\uparrow$ is a lifting of value x , then the result of $(\text{apply } f^\uparrow x^\uparrow)$ satisfies the correctness criteria (Definition 5.1) with respect to f x .*

PROOF. Given $v^\uparrow = (\text{apply } f^\uparrow x^\uparrow)$, we need to prove that given a configuration ρ , $v^\uparrow|_\rho = f(x^\uparrow|_\rho)$. According to the lifted-type and apply inference rules:

$$v^\uparrow = \{(f'(x'), pc) \mid (f', fpc) \in f^\uparrow, (x', xpc) \in x^\uparrow, pc = fpc \wedge xpc, \text{sat}(pc)\}$$

Let

$$c = \bigwedge_{e \in F} \begin{cases} e & \text{if } e \in \rho \\ \neg e & \text{otherwise} \end{cases}$$

This is the propositional formula representation of the configuration ρ , given a feature set F (Definition 2.2).

Since apply preserves the full coverage invariant (Theorem 5.3), $v = v^\uparrow|_\rho$ exists, such that $(v, vpc) \in v^\uparrow, \text{sat}(vpc \wedge c)$.

Then $v = v^\uparrow|_\rho = f(v^\uparrow|_\rho)$ (definition of lifted value indexing). $\square$

```

type      Val a = (a, PresenceCondition)
newtype Var t = Var [Val t]

mkVar :: PresenceCondition -> t -> Var t

mkVarT = mkVar ttPC

apply :: Var (a -> b) -> Var a -> Var b

instance Applicative Var where
  pure  = mkVarT
  (<*>) = apply

```

Fig. 9. Snippets of type and function definitions from the Haskell variability library.

6 IMPLEMENTATION

We implemented both shallow and deep lifting in Haskell¹, taking the subset of Haskell corresponding to PCF+ as input. Our lifting framework is comprised of two components: a library and a rewriting engine. The library includes a set of variability-aware polymorphic types, primitive operations, and helper functions used in the deep lifting of some Haskell constructs (e.g., conditional and pattern matching expressions). The Deep Rewriter engine is a source-to-source transformer that takes a program in our supported subset of Haskell, and generates its deep-lifted version. The overall architecture of our lifting framework (together with benchmarking and preprocessing components) is outlined in Fig. 10.

6.1 Lifted Types

The target language of our lifting engine is full-fledged Haskell, not just the PCF+-compliant subset. Fig. 9 shows snippets of type and function definitions from the Haskell variability library. We define a polymorphic variability-aware type `Var` as a list of pairs of values and presence conditions. We implement presence conditions as Binary Decision Diagrams (BDDs) using the CUDD library [Somenzi 1998]. Singleton `Var` values can be constructed using the `mkVar` function. The `mkVarT` function creates a singleton `Var` value with the True presence condition (`ttPC`).

Shallow-lifted functions are applied to arguments using the `apply` function, which implements the semantics of the `apply` inference rule. The library includes variants of `apply` that support different arities. In addition, the `Var` type instantiates the Haskell `Applicative` type class, so functions and operators available for that type class can be readily used on `Var` values.

Our implementation of `apply` checks presence condition conjuncts for satisfiability *before* applying the function to its argument. This way, if the conjunct is not satisfiable and the element is to be filtered out, we do not waste computational resources calculating a value that will be eliminated anyway. This is particularly important for computationally expensive functions.

6.2 Deep Rewriter

For deep lifting, we rewrite a Haskell program (in the PCF+ subset of Haskell) into a semantically equivalent variability-aware Haskell program. Our program rewriting engine is based on the

¹<https://github.com/ramyshahin/ProductLineAnalysis>

haskell-tools² library, which provides a programmatic interface to the Haskell Abstract Syntax Tree (AST). We traverse the AST of the input program, applying the rewrite rules defined in Sec. 4 to each syntactic construct that we support. An error is reported if the input program includes any unsupported syntactic constructs.

Because we use deep lifting polymorphic data types as well (Sec. 4.4), we need to keep track of term and expression types to make sure that the generated program conforms to type rules. Our implementation does not support lifting user-defined types at this point, but we expect the effort needed for that to be minimal. In general, lifted user-defined product types would use the specialized infrastructure we currently have for lifting pairs, and lifted user-defined sum types would use the infrastructure we have for lifting lists.

The program being rewritten might be using definitions exported by other modules. We provide a command-line option for specifying whether any module imports in the input program should be replaced by imports of deep-lifted modules instead. This allows for deep-lifting a code-base incrementally, one module at a time. If there is a dependency on a module that has not been deep-lifted, calls to functions from that module are shallow-lifted.

6.3 Laziness, Conditionals and Pattern Matching

Our theoretical treatment assumed Call-By-Name (CBN) semantics, where expressions in a control-flow branch are evaluated only when the branch is taken. In a variability-aware context, this is more complicated because multiple branches might be taken (for different subsets of lifted values), instead of only one. Filtering out values with unsatisfiable presence conditions before those values are computed ensures that expressions in a branch will only be applied to values where this branch is to be taken.

Haskell is a non-strict language (i.e., expression evaluation starts at the root of the expression AST, not the leaves), and different Haskell compilers and run-time systems implement non-strictness differently. Our Haskell implementation exhibited behaviors different from those in our theoretical model. In particular, some branches of conditional and pattern matching expressions were overactive (i.e., they were taken in cases where they should not). In some cases, this resulted in infinite loops when the overactive branch has a recursive call.

To mitigate these erroneous behaviors, we extended the implementation of lifted control-flow constructs (conditionals and pattern matching) with explicit context passing to branches. A *context* is the presence condition of values where a branch needs to be taken. Before evaluating expressions in a branch, values in those expressions are first *restricted* to the presence condition of the branch. Restriction here means removing atomic elements of those values that are outside the set of configurations defined by the branch presence condition. This ensures that even if a branch is overactive, its expressions will not be evaluated because their arguments will be turned into empty lifted values by restriction.

The lifted `if` and `case` expressions evaluate their guards first, and pass the presence conditions of guard values matching each of the branches to the respective branch. The signatures of lifted `if` and lifted `case` in our implementation are as follows:

```
liftedIf    :: Var Bool -> (PresenceCondition -> Var t)
            -> (PresenceCondition -> Var t) -> Var t
liftedCase  :: Var a -> (a -> Int)
            -> [PresenceCondition -> Var a -> Var b] -> Var b
```

where the second parameter of `liftedCase` is the splitter function, and the third parameter is a list of alternatives taking a `PresenceCondition` context as argument.

²<https://github.com/haskell-tools>

Table 1. Description of program analyses used in the evaluation.

Analysis	Description
Case Termination	Checking whether each non-empty case in a C-language switch statement ends with a break statement.
Dangling Switch	Checking if there is any dead-code (anything other than a declaration) between a switch statement and the first case or default.
Function Return Checker	Checking whether each non-void C function has at least one return statement.
Return Density	Calculates the average number of return statements per C function within a source file.
Goto Density	Calculates the average number of goto statements per label within a C source file.
Call Density	Calculates the average number of function calls per C function within a source file.

7.1 Evaluation Experiment Setup

We implemented six C-language program analyses, described in Table 1, in the PCF+ subset of Haskell. Each analysis takes a Control Flow Graph (CFG) of a C-language program as input, and either checks a property of the program or calculates a metric. Three of the analyses (Case Termination, Dangling Switch, and Function Return Checker) are adapted from [Von Rhein et al. 2018], and the others were designed by us.

The architecture of the benchmarking environment is outlined in Fig. 10. Each analysis is encapsulated in a brute-force harness, wrapped in a shallow-lifted version, and transformed using the Deep Rewriter into a deep-lifted version. The run-time performance of the three versions of each analysis is measured and compared using a benchmarking harness based on the Criterion³ Haskell microbenchmarking library. Criterion runs each benchmark multiple times (20 times by default), and estimates the mean running time using linear regression.

We applied each benchmark to a collection of 504 C-language source files from BusyBox⁴ version 1.18.5. These source files varied in size, complexity, and number of effective combinations. Each source file was transformed into a variational CFG file using the TypeChef variability-aware framework [Kästner et al. 2011]. We then parsed the CFG file into the input data structure of the analyses. All experiments were performed on a Quad-core Intel Core i7-6700 processor running at 3.4GHZ, with 16GB RAM, running 64-bit Ubuntu Linux (kernel version 4.15), and using the Glasgow Haskell Compiler version 8.6.5.

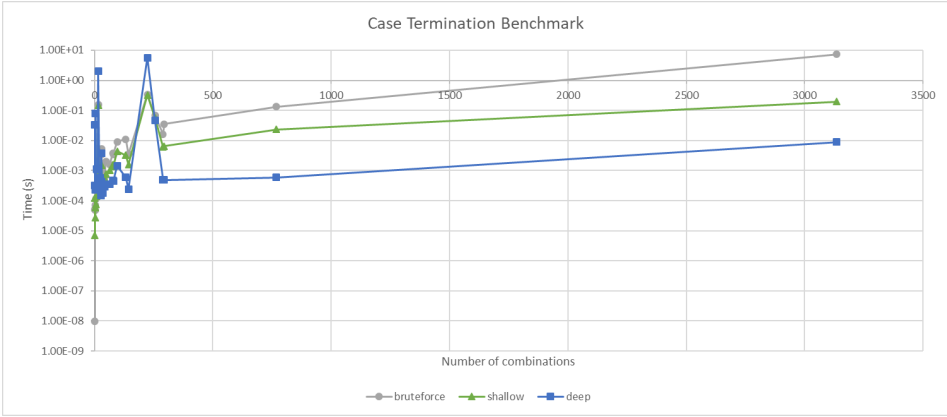
7.2 Evaluation Results

Results of our evaluation experiments are plotted in Fig. 11 and 12. We group the input source files based on their number of effective combinations, calculate the average run time for each group, and plot that against the number of effective combinations. The vertical axis of each of the six graphs is running time in seconds in log-scale.

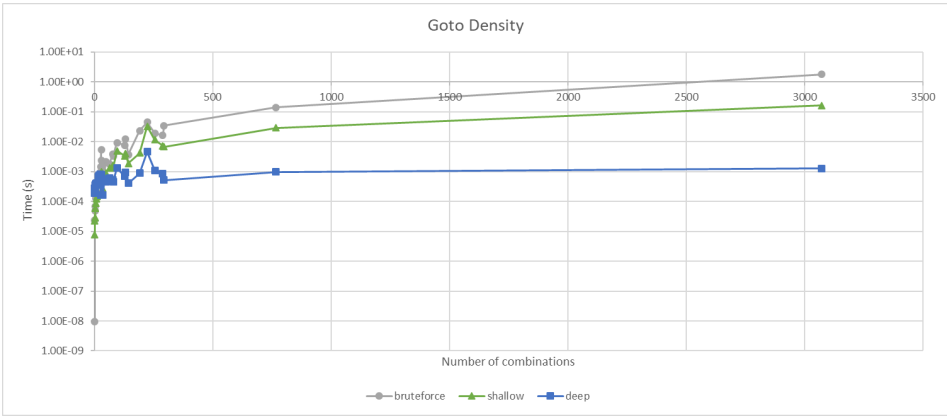
For a very small number of combinations, brute-force outperforms shallow lifting by more than an order of magnitude and outperforms deep lifting by two to three orders of magnitude in each of the benchmarks. However, as the number of effective combinations increases, shallow

³<http://www.serpentine.com/criterion/>

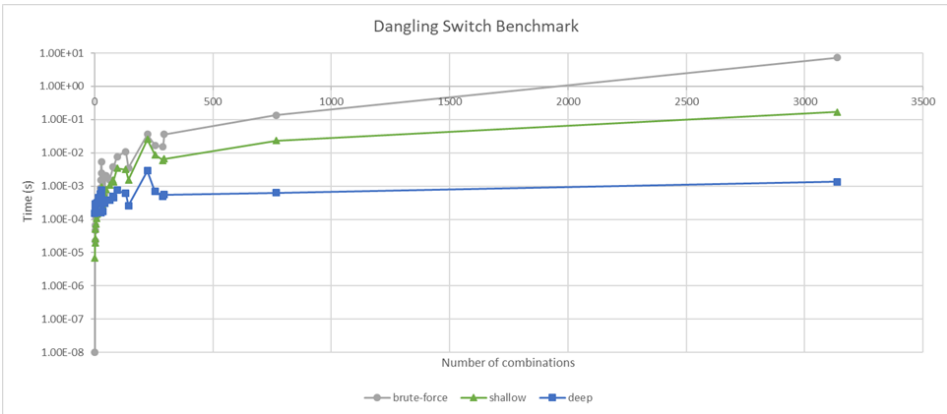
⁴<https://busybox.net/>



(a) Case Termination benchmark.

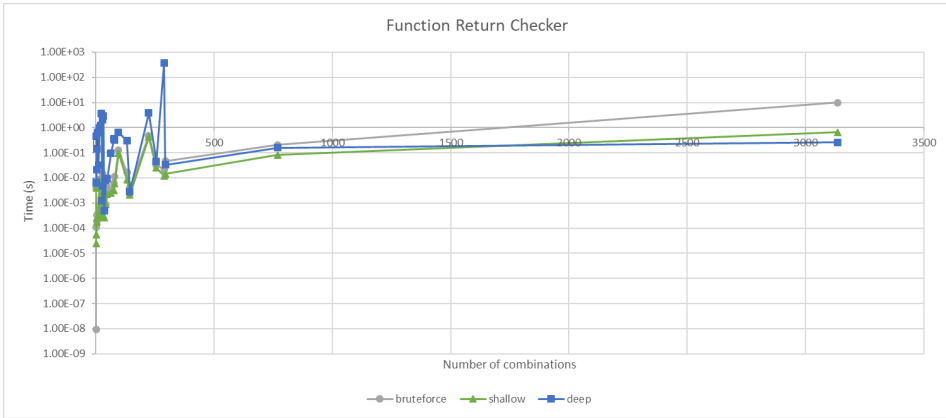


(b) Goto Density benchmark.



(c) Dangling Switch benchmark.

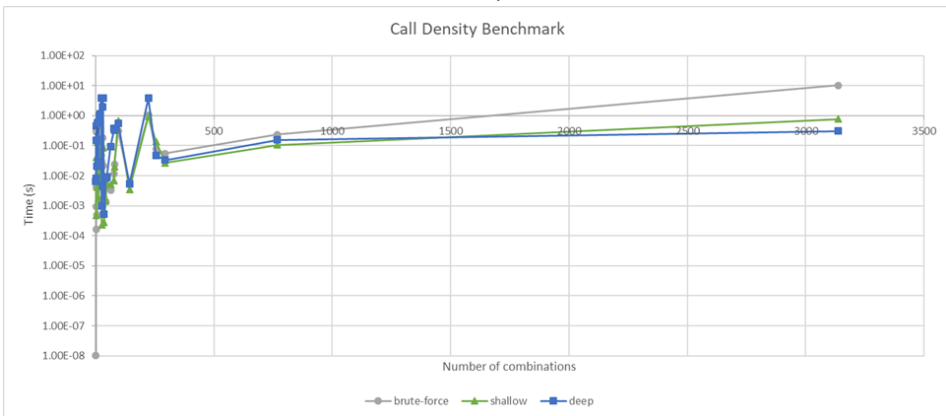
Fig. 11. Evaluation results on (a) case termination, (b) goto density per label, and (c) dangling switch. Horizontal axis is the number of effective product combinations in a source file, and vertical axis is average processing time of files with that number of configurations, in log-scale.



(a) Function Return Checker benchmark.



(b) Return Density benchmark.



(c) Call Density benchmark.

Fig. 12. Evaluation results on (a) function return checker, (b) return density per function, and (c) call density per function. Horizontal axis is the number of effective product combinations in a source file, and vertical axis is average processing time of files with that number of configurations, in log-scale.

lifting starts outperforming brute-force. The point at which deep lifting starts outperforming both brute-force and shallow lifting varies from one benchmark to another. In the Goto Density and Dangling Switch benchmarks, deep lifting takes over as the number of combinations go beyond 50. In Case Termination, deep lifting takes over at around 300 combinations. At 640 combinations, deep lifting is an order of magnitude faster than shallow lifting, and two orders of magnitude faster than brute-force in both benchmarks. However, in the Function Return Checker, Return Density, and Call Density benchmarks the performance of deep lifting fluctuates in the range from 10 to 300 combinations, and only slightly outperforms shallow lifting at around 3100 combinations (while outperforming brute-force by almost two orders of magnitude).

The input C files also vary in size and complexity, and this is likely the reason why we do not see a steady exponential growth in run time for brute-force analysis. The benchmark analyses also vary in terms of computational complexity, which explains the difference in performance patterns across them. For **RQ1**, although the overhead of deep lifting is evident throughout the range of effective combinations, this overhead grows slower than the exponential costs of brute-force and shallow lifting. The answer to **RQ2** varies across benchmarks, but overall we consistently see deep-lifting outperforming brute-force and shallow lifting as the number of effective combinations is in the hundreds.

7.3 Threats to Validity

Our benchmarks are all implementations of syntactic or control-flow analyses. Our framework is not specific to syntax or control flow analysis, but expanding to other categories of analysis requires infrastructure analogous to the TypeChef framework for variability-aware parsing and CFG generation. Also we cannot use existing analysis libraries directly because they typically use language features that we still do not support, so we had to write our own analyses.

As mentioned earlier, the C files being analyzed vary in size and complexity, and this skews the overall trend of results to some extent. This is more evident at the high end of combinations, as the number of source files per group is relatively small (sometimes only one), which negatively affects the generality of the results. However, since we care more about the overall trends of scalability as the number of combinations grows, we believe we can tolerate those skews.

8 DISCUSSION

Relaxing Total Coverage. The total coverage invariant states that a lifted value should cover the complete set of product variants defined by the feature model. Pragmatically, this over-constrains lifted values in some cases. For example, in Fig. 1 the argument b of type *int* only exists when feature **A** is not defined. Storing a representation of this argument by itself as a lifted value would violate the Full Coverage Invariant because it is only defined in a proper subset of valid products.

This invariant can be relaxed using an *Option* type (Maybe in Haskell) for the atomic elements of a lifted value. This way we can use *Nothing* as a valid value in products where the lifted value is not defined. This way the invariant is still strictly maintained, while allowing for incompleteness with respect to product space. Computing over *Nothing* values will still be a challenge though, as the correct behavior is application dependent in many cases.

Optimization and Canonicalization. The bigger the size of a lifted value (in terms of the number of atom values within it), the longer it takes (in terms of processing time) to process it. Recall that when applying a lifted function to a single lifted argument, we compute a cross-product of atom values, and check the satisfiability of the conjunction of presence conditions of each pair in the cross-product. This becomes a major performance bottleneck in any lifted program.

Of course, we cannot eliminate individual values from a lifted value. However, we can put a lifted value in *canonical* form by combining common values. For example, if a lifted value has two atom values (v_1, pc_1) and (v_2, pc_2) , and if v_1 and v_2 happen to be the same, then the two individual values can be collapsed into $(v_1, pc_1 \vee pc_2)$. Determining if v_1 and v_2 are the same can be done by checking their addresses in memory (a quick check) or, for types implementing an equality comparison operator, by checking if the actual values (and recursively those of inner fields) are the same (a more costly check).

Beyond SPLs. Lifting a functional program to a variable domain (where variability is labeled by presence conditions) is just one example of reusing the same algorithm in multiple computational contexts at once. Other examples include *probabilistic computing*, where a lifted value would have several atomic values each with a probability, and with the individual probabilities summing up to one. We can then directly map the different constructs specific to presence condition to those of probabilities. Assuming independent probabilities, conjunction of presence conditions is isomorphic to multiplication of probabilities. The disjointness and full coverage invariants are together isomorphic to the invariant that the sum of probabilities within a lifted value is always one.

9 RELATED WORK

Several software analyses have been re-implemented to be variability aware [Thüm et al. 2014]. For example, the TypeChef project [Kästner et al. 2012, 2011] implements variability aware parsers [Kästner et al. 2011] and type checkers [Kästner et al. 2012] for the Java and C languages. The SuperC project [Gazzillo and Grimm 2012] is another C language variability-aware parser. Some variability-aware control-flow and data-flow analyses have been implemented on top of TypeChef [Von Rhein et al. 2018]. These are all lifted analyses designed and implemented from scratch, as opposed to our approach of lifting an analysis *automatically*.

SPL^{Lift} [Bodden et al. 2013] extends IFDS [Reps et al. 1995] data flow analyses to product lines. Model checkers based on Featured Transition Systems [Classen et al. 2013] check temporal properties of transition systems where transitions can be labeled by presence conditions. These are two examples of SPL analyses that use almost the same single-product analyses on a lifted data representation. Our approach lifts algorithms together with the data structures to which they are applied.

Lifting a language instead of lifting individual analyses has been attempted for Datalog [Shahin and Chechik 2020; Shahin et al. 2019]. In this work, Datalog facts are assigned presence conditions, and a Datalog engine is modified to interpret and compute presence conditions together with inferring new facts. Our lifted values are quite similar in the sense that we associate presence conditions to atomic values, and compute them in lifted function application. We extend that notion in three ways. First, lifted values in our framework can hold multiple atomic values and presence conditions (subject to invariants). Second, and more importantly, we lift a Turing-complete language (PCF+) capable of expressing arbitrary analyses not expressible in Datalog. Finally, because of the computational expressiveness of PCF+, we did not need to modify a language interpreter. Instead, a source-to-source transformation of programs was sufficient.

Syntactic transformation systems have been suggested for lifting abstract interpretation analyses to SPLs [Midtgaard et al. 2015]. In this line of work, a systematic approach to lifting abstract interpretation analyses is outlined, together with correctness proofs. However, while this approach is systematic, it is not automated, requiring manually designed abstractions. In contrast, our approach is fully automated.

Empirical comparisons between brute force (product-based), variability-aware (family-based) and sampling techniques have also been conducted [Apel et al. 2013; Liebig et al. 2013]. Variability-aware analyses have been shown to outperform sampling, while not giving up configuration completeness.

Automated lifting of type-based analyses [Chen and Erwig 2014] to support annotative product lines is another attempt towards solving the SPL lifting problem. However, this approach only works if the analysis can be expressed as a type system. This restriction does not apply to our work.

Formalisms for developing variability-aware programs, e.g., the Choice Calculus [Erwig and Walkingshaw 2011], have been introduced. Those formalisms introduce variability constructs on top of conventional single-valued, deterministic formalisms and provide better language support for developing variability-aware analyses. However, to the best of our knowledge, automatic lifting of programs for those formalisms has not been attempted.

10 CONCLUSION

In this paper, we presented two approaches to automatic variability-aware lifting of software analyses written in a functional language (PCF+). A light-weight, black-box approach (shallow lifting) wraps an analysis in its variability-aware version. The second approach (deep lifting) is an automatic program-rewriting mechanism that translates an analysis program into a semantically equivalent variability-aware analysis.

We presented and discussed the program rewriting rules for different language constructs. We also presented formal correctness criteria of lifted programs, and showed how our approach conforms to it. We implemented our lifting framework in Haskell, and evaluated it on a set of program analyses. Our evaluation results show that shallow lifting outperforms brute-force analysis on Software Product Lines with relatively small number configurations. As the number of configurations increases, deep lifting outperforms both brute-force and shallow lifting.

For future work, we plan to extend our approach to supporting language constructs beyond PCF+. We also plan to work on improving the performance of lifted programs (lowering the overhead of presence condition computation). One more area we intend to explore is implementing and evaluating different policies for lifting data structures.

ACKNOWLEDGMENTS

We thank anonymous reviewers for their feedback, comments, and suggestions. This work was supported by General Motors and NSERC.

REFERENCES

- Alejandro Aguirre, Gilles Barthe, Marco Gaboardi, Deepak Garg, and Pierre-Yves Strub. 2017. A Relational Logic for Higher-Order Programs. *Proc. ACM Program. Lang.* 1, ICFP, Article 21 (Aug. 2017), 29 pages. <https://doi.org/10.1145/3110265>
- Sven Apel, Alexander von Rhein, Philipp Wendler, Armin Größlinger, and Dirk Beyer. 2013. Strategies for Product-line Verification: Case Studies and Experiments. In *Proc. of ICSE'13*. IEEE Press, 482–491.
- Michael Arntzenius and Neelakantan R. Krishnaswami. 2016. Datafun: A Functional Datalog. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming (Nara, Japan) (ICFP 2016)*. Association for Computing Machinery, New York, NY, USA, 214–227. <https://doi.org/10.1145/2951913.2951948>
- Eric Bodden, Tárzis Tolêdo, Márcio Ribeiro, Claus Brabrand, Paulo Borba, and Mira Mezini. 2013. SPLIFT: Statically Analyzing Software Product Lines in Minutes Instead of Years. In *Proc. of PLDI'13*. ACM, 355–364.
- Sheng Chen and Martin Erwig. 2014. Type-based Parametric Analysis of Program Families. In *Proc. of ICFP'14*. ACM, 39–51.
- Andreas Classen, Maxime Cordy, Pierre-Yves Schobbens, Patrick Heymans, Axel Legay, and Jean-Francois Raskin. 2013. Featured Transition Systems: Foundations for Verifying Variability-Intensive Systems and Their Application to LTL Model Checking. *IEEE Trans. Softw. Eng.* 39, 8 (Aug. 2013), 1069–1089.
- Paul Clements and Linda Northrop. 2001. *Software Product Lines: Practices and Patterns*. Addison-Wesley Professional.
- Krzysztof Czarnecki and Michal Antkiewicz. 2005. Mapping Features to Models: A Template Approach Based on Superimposed Variants. In *Proc. of GPCE'05*. 422–437.

- Martin Erwig and Eric Walkingshaw. 2011. The Choice Calculus: A Representation for Software Variation. *ACM Trans. Softw. Eng. Methodol.* 21, 1 (Dec. 2011), 6:1–6:27.
- Paul Gazzillo and Robert Grimm. 2012. SuperC: Parsing All of C by Taming the Preprocessor. In *Proc. of PLDI'12*. ACM, 323–334.
- Christian Kästner and Sven Apel. 2008. Integrating Compositional and Annotative Approaches for Product Line Engineering. In *Proc. of GPCE'08*. 35–40.
- Christian Kästner, Sven Apel, Thomas Thüm, and Gunter Saake. 2012. Type Checking Annotation-based Product Lines. *ACM Trans. Softw. Eng. Methodol.* 21, 3 (July 2012), 14:1–14:39.
- Christian Kästner, Paolo G. Giarrusso, Tillmann Rendel, Sebastian Erdweg, Klaus Ostermann, and Thorsten Berger. 2011. Variability-aware Parsing in the Presence of Lexical Macros and Conditional Compilation. In *Proc. of OOPSLA'11*. ACM, 805–824.
- G. A. Kavvos, Edward Morehouse, Daniel R. Licata, and Norman Danner. 2019. Recurrence Extraction for Functional Programs through Call-by-Push-Value. *Proc. ACM Program. Lang.* 4, POPL, Article 15 (Dec. 2019), 31 pages. <https://doi.org/10.1145/3371083>
- Sven Keidel and Sebastian Erdweg. 2019. Sound and Reusable Components for Abstract Interpretation. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 176 (Oct. 2019), 28 pages. <https://doi.org/10.1145/3360602>
- Jörg Liebig, Sven Apel, Christian Lengauer, Christian Kästner, and Michael Schulze. 2010. An Analysis of the Variability in Forty Preprocessor-Based Software Product Lines. In *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering - Volume 1* (Cape Town, South Africa) (ICSE '10). Association for Computing Machinery, New York, NY, USA, 105–114. <https://doi.org/10.1145/1806799.1806819>
- Jörg Liebig, Alexander von Rhein, Christian Kästner, Sven Apel, Jens Dörre, and Christian Lengauer. 2013. Scalable Analysis of Variable Software. In *Proc. of ESEC/FSE'13*. 81–91.
- Magnus Madsen, Ming-Ho Yee, and Ondrej Lhoták. 2016. From Datalog to Flix: A Declarative Language for Fixed Points on Lattices. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Santa Barbara, CA, USA) (PLDI '16). ACM, New York, NY, USA, 194–208. <https://doi.org/10.1145/2908080.2908096>
- Jan Midtgaard, Aleksandar S. Dimovski, Claus Brabrand, and Andrzej Wąsowski. 2015. Systematic Derivation of Correct Variability-aware Program Analyses. *Sci. Comput. Program.* 105, C (July 2015), 145–170.
- John C. Mitchell. 1996. *Foundations for Programming Languages*. The MIT Press.
- Sarah Nadi and Ric Holt. 2014. The Linux Kernel: A Case Study of Build System Variability. *J. Softw. Evol. Process* 26, 8 (Aug. 2014), 730–746. <https://doi.org/10.1002/smr.1595>
- Benjamin C. Pierce. 2002. *Types and Programming Languages* (1st ed.). The MIT Press.
- G.D. Plotkin. 1977. LCF considered as a programming language. *Theoretical Computer Science* 5, 3 (1977), 223 – 255. [https://doi.org/10.1016/0304-3975\(77\)90044-5](https://doi.org/10.1016/0304-3975(77)90044-5)
- Klaus Pohl, Günter Böckle, and Frank J. van der Linden. 2005. *Software Product Line Engineering: Foundations, Principles and Techniques*. Springer-Verlag, Berlin, Heidelberg.
- Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise Interprocedural Dataflow Analysis via Graph Reachability. In *Proc. of POPL'95*. ACM, 49–61.
- Julia Rubin and Marsha Chechik. 2012. Combining Related Products into Product Lines. In *Proc. of FASE'12*. LNCS, Vol. 7212. 285–300.
- Ulrich Schöpp. 2017. Defunctionalisation as Modular Closure Conversion. In *Proceedings of the 19th International Symposium on Principles and Practice of Declarative Programming* (Namur, Belgium) (PPDP '17). Association for Computing Machinery, New York, NY, USA, 175–186. <https://doi.org/10.1145/3131851.3131868>
- Ramy Shahin and Marsha Chechik. 2020. Variability-Aware Datalog. In *Practical Aspects of Declarative Languages*, Ekaterina Komendantskaya and Yanhong Annie Liu (Eds.). Springer International Publishing, 213–221.
- Ramy Shahin, Marsha Chechik, and Rick Salay. 2019. Lifting Datalog-based Analyses to Software Product Lines. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Tallinn, Estonia) (ESEC/FSE 2019). ACM, New York, NY, USA, 39–49.
- Fabio Somenzi. 1998. CUDD: CU Decision Diagram Package Release 2.2.0. (06 1998).
- Thomas Thüm, Sven Apel, Christian Kästner, Ina Schaefer, and Gunter Saake. 2014. A Classification and Survey of Analysis Strategies for Software Product Lines. *ACM Comput. Surv.* 47, 1, Article 6 (June 2014), 45 pages.
- Alexander Von Rhein, Jörg Liebig, Andreas Janker, Christian Kästner, and Sven Apel. 2018. Variability-Aware Static Analysis at Scale: An Empirical Study. *ACM Trans. Softw. Eng. Methodol.* 27, 4, Article 18 (Nov. 2018), 33 pages. <https://doi.org/10.1145/3280986>